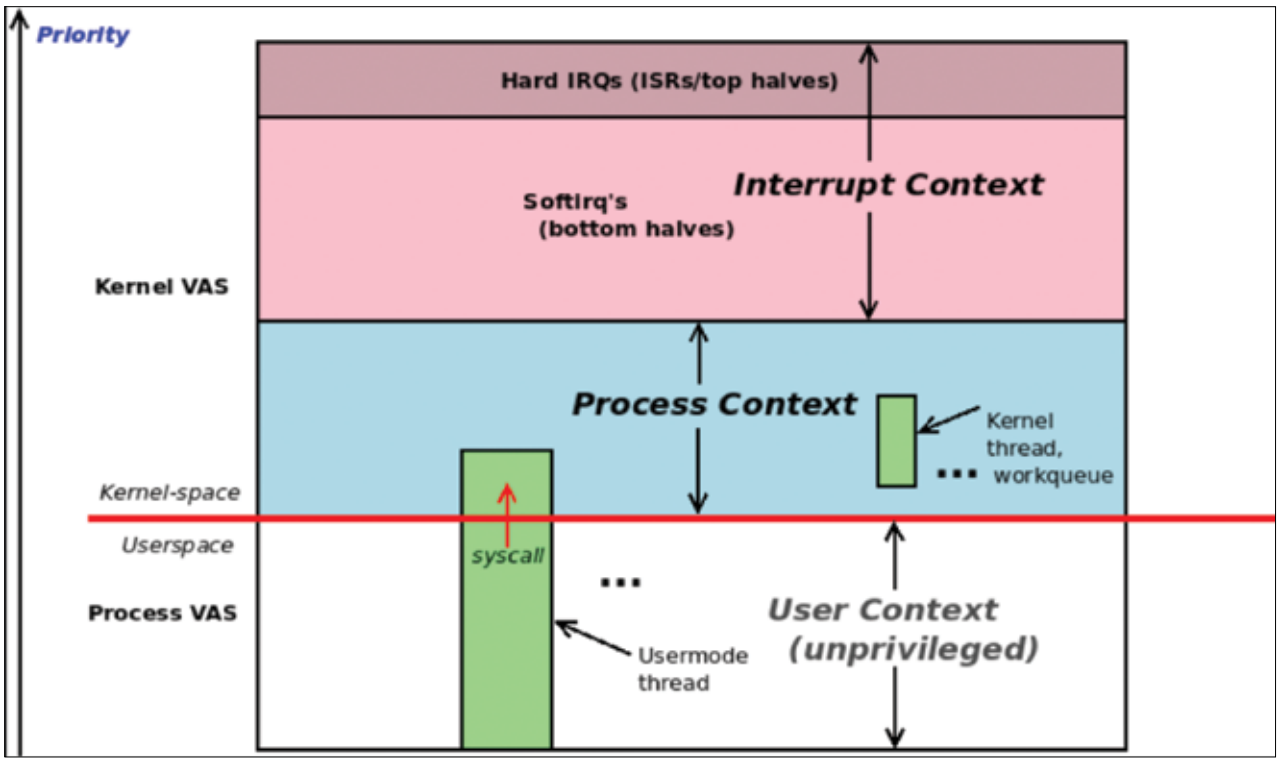


Figure 6.1: Conceptual diagram showing unprivileged user-mode execution and privileged kernel-mode execution with both process and interrupt contexts

- **Process context:** The kernel is entered from a system call or processor *exception* (such as a page fault) and kernel code is executed, kernel data worked upon; it's synchronous (top down).
- **Interrupt context:** The kernel is entered from a peripheral chip's hardware interrupt and kernel code is executed, kernel data worked upon; it's asynchronous (bottom up).

Figure 6.1 shows the conceptual view: user-mode processes and threads execute in unprivileged user context; the user mode thread can switch to privileged kernel mode by issuing a *system call*. The diagram also shows us that pure *kernel threads* exist as well within Linux; they're very similar to user-mode threads, with the key difference that they only execute in kernel space; they cannot even see the user VAS. A synchronous switch to kernel mode via a system call (or processor exception) has the task now running kernel code in process context. (Kernel threads too run kernel code in process context.) Hardware interrupts, though, are a different ball game – they cause execution to asynchronously enter the kernel; the code they execute (typically a device driver's interrupt handler) runs in the so-called *interrupt context.*

Figure 6.1 shows more details – interrupt context top and bottom halves, kernel threads and workqueues; we request you to have some patience, we'll cover all this and much more in later chapters.

Further on in the book, we shall show you how exactly you can check in which context your kernel code is currently running. Read on! END

**Note:** The book's source repositories are publicly available on GitHub, as indicated below.

Linux Kernel Programming (Part 1): A comprehensive guide to kernel internals, writing kernel modules, and kernel synchronization; *https://github.com/PacktPublishing/Linux-Kernel-Programming*

Linux Kernel Programming (Part 2): Learn to work with 'misc' class char drivers, user-kernel interfaces, manage peripheral I/O and hardware interrupts; *https://github.com/PacktPublishing/Linux-Kernel-Programming-Part-2*

**By: Kaiwan N. Billimoria**

The author is a founder at kaiwanTECH (Designer Graphix) and has spent long days and nights writing this book. His Linux passion feeds well into his passion for teaching these topics to engineers, which he has done for well over two decades now.